Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```sql
CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    DOB DATE,
    Balance NUMBER,
    LastModified DATE
);
CREATE TABLE Loans (
    LoanID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    LoanAmount NUMBER,
    InterestRate NUMBER,
    StartDate DATE,
    EndDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
INSERT INTO Customers
VALUES (1,'John Doe',DATE '1955-05-15',1000,SYSDATE);

INSERT INTO Customers
VALUES (2,'Jane Smith',DATE '1995-07-20',1500,SYSDATE);

INSERT INTO Loans
VALUES (1,1,5000,5,SYSDATE,ADD_MONTHS(SYSDATE,60));
```

```sql
COMMIT;
SELECT * FROM Customers;
SELECT * FROM Loans;
BEGIN
    FOR c IN (
        SELECT CustomerID,
               FLOOR(MONTHS_BETWEEN(SYSDATE, DOB)/12) Age
        FROM Customers
    )
    LOOP
        IF c.Age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE CustomerID = c.CustomerID;
        END IF;
    END LOOP;

    COMMIT;
END;
```

Download Execution time: 0.155 seconds

	CUSTOMERID	NAME	DOB	BALANCE	LASTMODIFIED
1	1	John Doe	5/15/1955, 12:00:0(	1000	7/4/2026, 3:12:33 P
2	2	Jane Smith	7/20/1995, 12:00:0(	1500	7/4/2026, 3:12:33 P

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over $10,000.

```
ALTER TABLE Customers
ADD IsVIP VARCHAR2(5);
BEGIN
    FOR c IN (
        SELECT CustomerID, Balance
        FROM Customers
    )
    LOOP
        IF c.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP = 'TRUE'
            WHERE CustomerID = c.CustomerID;
        ELSE
            UPDATE Customers
            SET IsVIP = 'FALSE'
            WHERE CustomerID = c.CustomerID;
        END IF;
    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('VIP status updated successfully.');

END;
/
INSERT INTO Customers
VALUES (1, 'John', DATE '1985-05-15', 12000, SYSDATE, NULL);

INSERT INTO Customers
VALUES (2, 'Jane', DATE '1990-07-20', 8000, SYSDATE, NULL);

COMMIT;
SELECT CustomerID, Name, Balance, IsVIP
FROM Customers;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.001 seconds

	D	NAME	DOB	BALANCE	LASTMODIFIED	ISVIP
1	1	John Doe	5/15/1955, 12:00:0(	1000	7/4/2026, 3:12:33 P	FALSE
2	2	Jane Smith	7/20/1995, 12:00:0(	1500	7/4/2026, 3:12:33 P	FALSE

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
COMMIT;
BEGIN
    FOR l IN (
        SELECT c.Name,
               l.LoanID,
               l.EndDate
        FROM Customers c
        JOIN Loans l
        ON c.CustomerID = l.CustomerID
        WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Reminder: Dear ' || l.Name ||
            ', your Loan ID ' || l.LoanID ||
            ' is due on ' || TO_CHAR(l.EndDate, 'DD-MON-YYYY')
        );
    END LOOP;
END;
/
```

```
Reminder: Dear John Doe, your Loan ID 101 is due on 14-JUL-2026


PL/SQL procedure successfully completed.
```

Exercise 2: Error Handling

Scenario 1: Handle exceptions during fund transfers between accounts.

- **Question:** Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

```sql
CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    AccountType VARCHAR2(20),
    Balance NUMBER,
    LastModified DATE
);
INSERT INTO Accounts
VALUES (1,101,'Savings',5000,SYSDATE);

INSERT INTO Accounts
VALUES (2,102,'Savings',2000,SYSDATE);

COMMIT;
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
    p_FromAccount NUMBER,
    p_ToAccount NUMBER,
    p_Amount NUMBER
)
IS
    v_Balance NUMBER;
BEGIN
    -- Get source account balance
    SELECT Balance
    INTO v_Balance
    FROM Accounts
```

```
    WHERE AccountID = p_FromAccount;

    -- Check if sufficient balance exists
    IF v_Balance < p_Amount THEN
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient Funds');
    END IF;

    -- Deduct amount from source account
    UPDATE Accounts
    SET Balance = Balance - p_Amount
    WHERE AccountID = p_FromAccount;

    -- Add amount to destination account
    UPDATE Accounts
    SET Balance = Balance + p_Amount
    WHERE AccountID = p_ToAccount;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Funds transferred successfully.');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Transfer Failed: ' || SQLERRM);
END;
/
BEGIN
    SafeTransferFunds(1,2,1000);
END;
/
```

```
SQL> BEGIN
         SafeTransferFunds(1,2,1000);
     END;


Funds transferred successfully.


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.012
```

Scenario 2: Manage errors when updating employee salaries.

- **Question:** Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

```
CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_EmployeeID NUMBER,
    p_Percentage NUMBER
)
IS
BEGIN

    UPDATE Employees
    SET Salary = Salary + (Salary * p_Percentage / 100)
    WHERE EmployeeID = p_EmployeeID;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Employee ID does not exist.');

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
/
BEGIN
    UpdateSalary(1,10);
END;
/
```

```
Elapsed: 00:00:00.008


SQL> BEGIN
         UpdateSalary(1,10);
     END;


Salary updated successfully.


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.013
```

Scenario 3: Ensure data integrity when adding a new customer.

- **Question:** Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```sql
CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_CustomerID NUMBER,
    p_Name VARCHAR2,
    p_DOB DATE,
    p_Balance NUMBER
)
IS
BEGIN
    INSERT INTO Customers
    (
        CustomerID,
        Name,
        DOB,
        Balance,
        LastModified
    )
    VALUES
    (
        p_CustomerID,
        p_Name,
        p_DOB,
        p_Balance,
        SYSDATE
    );

    COMMIT;
```

```sql
34
35          DBMS_OUTPUT.PUT_LINE('Customer added successfully.');
36
37      EXCEPTION
38          WHEN DUP_VAL_ON_INDEX THEN
39              ROLLBACK;
40              DBMS_OUTPUT.PUT_LINE('Error: Customer ID already exists.');
41
42          WHEN OTHERS THEN
43              ROLLBACK;
44              DBMS_OUTPUT.PUT_LINE(SQLERRM);
45      END;
46      /
47      BEGIN
48          AddNewCustomer(
49              2,
50              'Jane Smith',
51              DATE '1998-08-20',
52              15000S
53          );
54      END;
55      /
```

```
SQL> BEGIN
         AddNewCustomer(
             2,
             'Jane Smith',...
Show more...


Error: Customer ID already exists.


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.008
```

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```sql
CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    AccountType VARCHAR2(20),
    Balance NUMBER,
    LastModified DATE
);
INSERT INTO Accounts
VALUES (1, 101, 'Savings', 10000, SYSDATE);

INSERT INTO Accounts
VALUES (2, 102, 'Savings', 5000, SYSDATE);

INSERT INTO Accounts
VALUES (3, 103, 'Checking', 8000, SYSDATE);

COMMIT;
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN

    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;
```

```
27
28      DBMS_OUTPUT.PUT_LINE('Monthly interest applied successfully.');
29
30  EXCEPTION
31      WHEN OTHERS THEN
32          ROLLBACK;
33          DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
34  END;
35  /
36  BEGIN
37      ProcessMonthlyInterest;
38  END;
39  /
```

```
Elapsed: 00:00:00.019

SQL> BEGIN
          ProcessMonthlyInterest;
      END;


Monthly interest applied successfully.


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010
```

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```sql
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_Department VARCHAR2,
    p_BonusPercent NUMBER
)
IS
BEGIN

    UPDATE Employees
    SET Salary = Salary + (Salary * p_BonusPercent / 100)
    WHERE Department = p_Department;

    IF SQL%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No employees found in the department.');
    ELSE
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Employee bonus updated successfully.');
    END IF;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
BEGIN
    UpdateEmployeeBonus('IT', 10);
```

```
SQL> BEGIN
         UpdateEmployeeBonus('IT', 10);
     END;


Employee bonus updated successfully.


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014
```

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```sql
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_FromAccount NUMBER,
    p_ToAccount NUMBER,
    p_Amount NUMBER
)
IS
    v_Balance NUMBER;
BEGIN

    -- Get balance of source account
    SELECT Balance
    INTO v_Balance
    FROM Accounts
    WHERE AccountID = p_FromAccount;

    -- Check sufficient balance
    IF v_Balance < p_Amount THEN
        DBMS_OUTPUT.PUT_LINE('Insufficient balance.');
    ELSE
        -- Deduct amount from source account
        UPDATE Accounts
        SET Balance = Balance - p_Amount
        WHERE AccountID = p_FromAccount;

        -- Add amount to destination account
        UPDATE Accounts
        SET Balance = Balance + p_Amount
        WHERE AccountID = p_ToAccount;

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Funds transferred successfully.');
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Source account not found.');

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
BEGIN
    TransferFunds(1, 2, 2000);
END;
/
```

```
SQL> BEGIN
        TransferFunds(1, 2, 2000);
     END;

Funds transferred successfully.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.006
```

Exercise 4: Function

Scenario 1: Calculate the age of customers for eligibility checks.

- **Question:** Write a function CalculateAge that takes a customer's date of birth as input and returns their age in years.

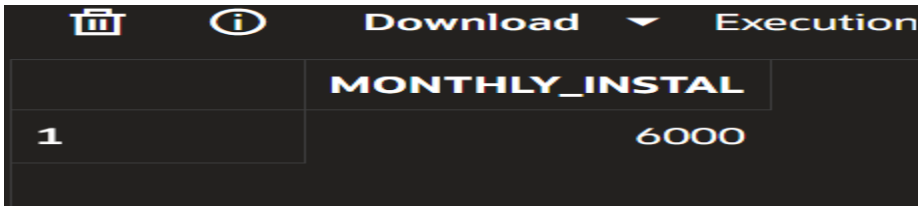
```sql
CREATE OR REPLACE FUNCTION CalculateAge(
    p_DOB DATE
)
RETURN NUMBER
IS
    v_Age NUMBER;
BEGIN
    v_Age := FLOOR(MONTHS_BETWEEN(SYSDATE, p_DOB) / 12);
    RETURN v_Age;
END;
/
SELECT CalculateAge(DATE '1998-05-15') AS Age
FROM DUAL;
```

	AGE
1	28

Scenario 2: The bank needs to compute the monthly installment for a loan.

- **Question:** Write a function **CalculateMonthlyInstallment** that takes the loan amount, interest rate, and loan duration in years as input and returns the monthly installment amount.

```sql
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(
    p_LoanAmount NUMBER,
    p_InterestRate NUMBER,
    p_Years NUMBER
)
RETURN NUMBER
IS
    v_MonthlyInstallment NUMBER;
BEGIN
    -- Simple Interest Formula
    v_MonthlyInstallment :=
        (p_LoanAmount + (p_LoanAmount * p_InterestRate * p_Years / 100))
        / (p_Years * 12);

    RETURN v_MonthlyInstallment;
END;
/
SELECT CalculateMonthlyInstallment(120000, 10, 2) AS Monthly_Installment
FROM DUAL;
```



Scenario 3: Check if a customer has sufficient balance before making a transaction.

- **Question:** Write a function **HasSufficientBalance** that takes an account ID and an amount as input and returns a boolean indicating whether the account has at least the specified amount.

```sql
CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    AccountType VARCHAR2(20),
    Balance NUMBER,
    LastModified DATE
);
INSERT INTO Accounts
VALUES (1, 101, 'Savings', 10000, SYSDATE);

INSERT INTO Accounts
VALUES (2, 102, 'Savings', 5000, SYSDATE);

COMMIT;
CREATE OR REPLACE FUNCTION HasSufficientBalance(
    p_AccountID NUMBER,
    p_Amount NUMBER
)
RETURN BOOLEAN
IS
    v_Balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_Balance
    FROM Accounts
    WHERE AccountID = p_AccountID;
    IF v_Balance >= p_Amount THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END;
/
DECLARE
    v_Result BOOLEAN;
BEGIN
    v_Result := HasSufficientBalance(1, 5000);

    IF v_Result THEN
        DBMS_OUTPUT.PUT_LINE('Sufficient Balance');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
    END IF;
END;
/
```

```
Insufficient Balance

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.007
```

Exercise 5: Triggers

Scenario 1: Automatically update the last modified date when a customer's record is updated.

- **Question:** Write a trigger **UpdateCustomerLastModified** that updates the LastModified column of the Customers table to the current date whenever a customer's record is updated.

```sql
CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    DOB DATE,
    Balance NUMBER,
    LastModified DATE
);
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE
ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/
UPDATE Customers
SET Balance = 15000
WHERE CustomerID = 1;

COMMIT;

```

```
1 row updated.

Elapsed: 00:00:00.005

SQL> COMMIT


Commit complete.

Elapsed: 00:00:00.002
```

Scenario 2: Maintain an audit log for all transactions.

- **Question:** Write a trigger **LogTransaction** that inserts a record into an AuditLog table whenever a transaction is inserted into the Transactions table.

```sql
CREATE TABLE AuditLog (
    LogID NUMBER PRIMARY KEY,
    TransactionID NUMBER,
    Action VARCHAR2(50),
    LogDate DATE
);
CREATE TABLE Transactions (
    TransactionID NUMBER PRIMARY KEY,
    AccountID NUMBER,
    TransactionDate DATE,
    Amount NUMBER,
    TransactionType VARCHAR2(20)
);
CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT
ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog
    VALUES (
        :NEW.TransactionID,
        :NEW.TransactionID,
        'Transaction Inserted',
        SYSDATE
    );
END;
```

```sql
/
INSERT INTO Transactions
VALUES (1,101,SYSDATE,5000,'Deposit');

COMMIT;
```

```
1 row inserted.
Elapsed: 00:00:00.032

SQL> COMMIT


Commit complete.
Elapsed: 00:00:00.001
```

Scenario 3: Enforce business rules on deposits and withdrawals.

- **Question:** Write a trigger **CheckTransactionRules** that ensures withdrawals do not exceed the balance and deposits are positive before inserting a record into the Transactions table.

```sql
CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    AccountType VARCHAR2(20),
    Balance NUMBER,
    LastModified DATE
);
INSERT INTO Accounts
VALUES (101, 1, 'Savings', 10000, SYSDATE);

COMMIT;
CREATE TABLE Transactions (
    TransactionID NUMBER PRIMARY KEY,
    AccountID NUMBER,
    TransactionDate DATE,
    Amount NUMBER,
    TransactionType VARCHAR2(20)
);
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT
ON Transactions
FOR EACH ROW
DECLARE
    v_Balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_Balance
    FROM Accounts
    WHERE AccountID = :NEW.AccountID;
    IF :NEW.TransactionType = 'Withdrawal' THEN
        IF :NEW.Amount > v_Balance THEN
            RAISE_APPLICATION_ERROR(-20001,
            'Withdrawal exceeds available balance.');
        END IF;
    ELSIF :NEW.TransactionType = 'Deposit' THEN
        IF :NEW.Amount <= 0 THEN
            RAISE_APPLICATION_ERROR(-20002,
            'Deposit amount must be greater than zero.');
        END IF;
    END IF;
END;
/
INSERT INTO Transactions
VALUES (1, 101, SYSDATE, 2000, 'Deposit');
```

Exercise 6: Cursors

Scenario 1: Generate monthly statements for all customers.

- **Question:** Write a PL/SQL block using an explicit cursor **GenerateMonthlyStatements** that retrieves all transactions for the current month and prints a statement for each customer.

```
DECLARE
    CURSOR GenerateMonthlyStatements IS
        SELECT t.TransactionID,
               c.Name,
               t.TransactionDate,
               t.Amount,
               t.TransactionType
        FROM Customers c
        JOIN Accounts a
          ON c.CustomerID = a.CustomerID
        JOIN Transactions t
          ON a.AccountID = t.AccountID
        WHERE EXTRACT(MONTH FROM t.TransactionDate) = EXTRACT(MONTH FROM SYSDATE)
          AND EXTRACT(YEAR FROM t.TransactionDate) = EXTRACT(YEAR FROM SYSDATE);

    v_TransactionID Transactions.TransactionID%TYPE;
    v_Name Customers.Name%TYPE;
    v_Date Transactions.TransactionDate%TYPE;
    v_Amount Transactions.Amount%TYPE;
    v_Type Transactions.TransactionType%TYPE;

BEGIN
    OPEN GenerateMonthlyStatements;

    LOOP
        FETCH GenerateMonthlyStatements
            INTO v_TransactionID, v_Name, v_Date, v_Amount, v_Type;

        EXIT WHEN GenerateMonthlyStatements%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Customer: ' || v_Name ||
            ', Transaction ID: ' || v_TransactionID ||
            ', Date: ' || TO_CHAR(v_Date,'DD-MON-YYYY') ||
            ', Type: ' || v_Type ||
            ', Amount: ' || v_Amount
        );
    END LOOP;

    CLOSE GenerateMonthlyStatements;
END;
/
```

```
SQL> DECLARE
         CURSOR GenerateMonthlyStatements IS
             SELECT t.TransactionID,
                    c.Name,...
Show more...


Customer: John Doe, Transaction ID: 1, Date: 04-JUL-2026, Type: Deposit, Amount: 5000


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.029
```

Scenario 2: Apply annual fee to all accounts.

- **Question:** Write a PL/SQL block using an explicit cursor **ApplyAnnualFee** that deducts an annual maintenance fee from the balance of all accounts.

```sql
CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    AccountType VARCHAR2(20),
    Balance NUMBER,
    LastModified DATE
);
INSERT INTO Accounts
VALUES (1,101,'Savings',10000,SYSDATE);

INSERT INTO Accounts
VALUES (2,102,'Checking',8000,SYSDATE);

INSERT INTO Accounts
VALUES (3,103,'Savings',12000,SYSDATE);

COMMIT;
DECLARE
    CURSOR ApplyAnnualFee IS
        SELECT AccountID, Balance
        FROM Accounts
        FOR UPDATE;

    v_AccountID Accounts.AccountID%TYPE;
    v_Balance Accounts.Balance%TYPE;

```

```
    v_Fee NUMBER := 500;

BEGIN
    OPEN ApplyAnnualFee;

    LOOP
        FETCH ApplyAnnualFee
        INTO v_AccountID, v_Balance;

        EXIT WHEN ApplyAnnualFee%NOTFOUND;

        UPDATE Accounts
        SET Balance = Balance - v_Fee
        WHERE CURRENT OF ApplyAnnualFee;

        DBMS_OUTPUT.PUT_LINE('Annual fee deducted from Account ID: ' || v_AccountID);

    END LOOP;

    CLOSE ApplyAnnualFee;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Annual maintenance fee applied successfully.');

```

```
Annual fee deducted from Account ID: 101
Annual fee deducted from Account ID: 1
Annual fee deducted from Account ID: 2
Annual fee deducted from Account ID: 3
Annual maintenance fee applied successfully.


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010
```

Scenario 3: Update the interest rate for all loans based on a new policy.

- **Question:** Write a PL/SQL block using an explicit cursor **UpdateLoanInterestRates** that fetches all loans and updates their interest rates based on the new policy.

```sql
CREATE TABLE Loans (
    LoanID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    LoanAmount NUMBER,
    InterestRate NUMBER,
    StartDate DATE,
    EndDate DATE
);
INSERT INTO Loans
VALUES (1,101,50000,8,SYSDATE,ADD_MONTHS(SYSDATE,12));

INSERT INTO Loans
VALUES (2,102,70000,9,SYSDATE,ADD_MONTHS(SYSDATE,24));

INSERT INTO Loans
VALUES (3,103,100000,10,SYSDATE,ADD_MONTHS(SYSDATE,36));

COMMIT;
DECLARE
    CURSOR UpdateLoanInterestRates IS
        SELECT LoanID, InterestRate
        FROM Loans
        FOR UPDATE;

    v_LoanID Loans.LoanID%TYPE;
    v_InterestRate Loans.InterestRate%TYPE;
```

```sql
BEGIN
    OPEN UpdateLoanInterestRates;

    LOOP
        FETCH UpdateLoanInterestRates
        INTO v_LoanID, v_InterestRate;

        EXIT WHEN UpdateLoanInterestRates%NOTFOUND;

        UPDATE Loans
        SET InterestRate = InterestRate + 1
        WHERE CURRENT OF UpdateLoanInterestRates;

        DBMS_OUTPUT.PUT_LINE('Updated Loan ID: ' || v_LoanID);

    END LOOP;

    CLOSE UpdateLoanInterestRates;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Interest rates updated successfully.');

END;
```

```
Updated Loan ID: 101
Updated Loan ID: 102
Updated Loan ID: 1
Interest rates updated successfully.


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014
```

Exercise 7: Packages

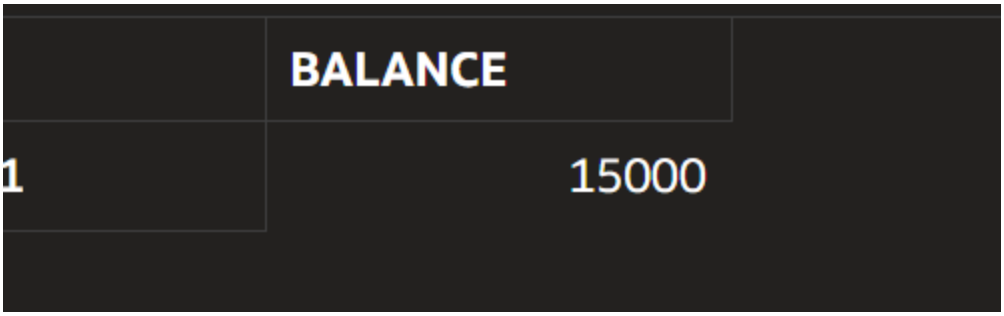
Scenario 1: Group all customer-related procedures and functions into a package.

- **Question:** Create a package **CustomerManagement** with procedures for adding a new customer, updating customer details, and a function to get customer balance.

```sql
CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

    PROCEDURE AddCustomer(
        p_CustomerID NUMBER,
        p_Name VARCHAR2,
        p_DOB DATE,
        p_Balance NUMBER
    )
    IS
    BEGIN
        INSERT INTO Customers
        (CustomerID, Name, DOB, Balance, LastModified)
        VALUES
        (p_CustomerID, p_Name, p_DOB, p_Balance, SYSDATE);
    END AddCustomer;
    PROCEDURE UpdateCustomer(
        p_CustomerID NUMBER,
        p_Name VARCHAR2,
        p_Balance NUMBER
    )
    IS
    BEGIN
        UPDATE Customers
        SET Name = p_Name,
            Balance = p_Balance,
            LastModified = SYSDATE
```

```
        WHERE CustomerID = p_CustomerID;
    END UpdateCustomer;
    FUNCTION GetCustomerBalance(
        p_CustomerID NUMBER
    )
    RETURN NUMBER
    IS
        v_Balance NUMBER;
    BEGIN
        SELECT Balance
        INTO v_Balance
        FROM Customers
        WHERE CustomerID = p_CustomerID;

        RETURN v_Balance;
    END GetCustomerBalance;

END CustomerManagement;
/
BEGIN
    CustomerManagement.AddCustomer(
        1,
        'John Doe',
```

```
    CustomerManagement.AddCustomer(
        1,
        'John Doe',
        DATE '1998-05-15',
        10000
    );

    COMMIT;
END;
/

BEGIN
    CustomerManagement.UpdateCustomer(
        1,
        'John Smith',
        15000
    );

    COMMIT;
END;
/
SELECT CustomerManagement.GetCustomerBalance(1) AS Balance
FROM DUAL;
SELECT * FROM Customers;
```



Scenario 2: Create a package to manage employee data.

- **Question:** Write a package **EmployeeManagement** with procedures to hire new employees, update employee details, and a function to calculate annual salary.

```
END EmployeeManagement;
/
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
    PROCEDURE HireEmployee(
        p_EmployeeID NUMBER,
        p_Name VARCHAR2,
        p_Position VARCHAR2,
        p_Salary NUMBER,
        p_Department VARCHAR2
    )
    IS
    BEGIN
        INSERT INTO Employees
        (EmployeeID, Name, Position, Salary, Department, HireDate)
        VALUES
        (p_EmployeeID, p_Name, p_Position, p_Salary, p_Department, SYSDATE);
    END HireEmployee;
    PROCEDURE UpdateEmployee(
        p_EmployeeID NUMBER,
        p_Name VARCHAR2,
        p_Position VARCHAR2,
        p_Salary NUMBER,
        p_Department VARCHAR2
    )
    IS
```

```
    BEGIN
        UPDATE Employees
        SET Name = p_Name,
            Position = p_Position,
            Salary = p_Salary,
            Department = p_Department
        WHERE EmployeeID = p_EmployeeID;
    END UpdateEmployee;
    FUNCTION CalculateAnnualSalary(
        p_EmployeeID NUMBER
    ) RETURN NUMBER
    IS
        v_Salary NUMBER;
    BEGIN
        SELECT Salary
        INTO v_Salary
        FROM Employees
        WHERE EmployeeID = p_EmployeeID;

        RETURN v_Salary * 12;
    END CalculateAnnualSalary;

END EmployeeManagement;
/
BEGIN
    EmployeeManagement.HireEmployee(
        1,
        'Alice Johnson',
        'Manager',
        50000,
        'HR'
    );

    COMMIT;
END;
/
BEGIN
    EmployeeManagement.UpdateEmployee(
        1,
        'Alice Johnson',
        'Senior Manager',
        60000,
        'HR'
    );

    COMMIT;
END;
/
SELECT EmployeeManagement.CalculateAnnualSalary(1) AS Annual_Salary
FROM DUAL;
```

	ANNUAL_SALARY
1	720000

Scenario 3: Group all account-related operations into a package.

- **Question:** Create a package **AccountOperations** with procedures for opening a new account, closing an account, and a function to get the total balance of a customer across all accounts.

```
CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    AccountType VARCHAR2(20),
    Balance NUMBER,
    LastModified DATE
);
CREATE OR REPLACE PACKAGE AccountOperations AS

    PROCEDURE OpenAccount(
        p_AccountID NUMBER,
        p_CustomerID NUMBER,
        p_AccountType VARCHAR2,
        p_Balance NUMBER
    );
    PROCEDURE CloseAccount(
        p_AccountID NUMBER
    );
    FUNCTION GetTotalBalance(
        p_CustomerID NUMBER
    ) RETURN NUMBER;
END AccountOperations;
/
CREATE OR REPLACE PACKAGE BODY AccountOperations AS
    PROCEDURE OpenAccount(
        p_AccountID NUMBER,
```

```
        p_CustomerID NUMBER,
        p_AccountType VARCHAR2,
        p_Balance NUMBER
    )
    IS
    BEGIN
        INSERT INTO Accounts
        (AccountID, CustomerID, AccountType, Balance, LastModified)
        VALUES
        (p_AccountID, p_CustomerID, p_AccountType, p_Balance, SYSDATE);
    END OpenAccount;
    PROCEDURE CloseAccount(
        p_AccountID NUMBER
    )
    IS
    BEGIN
        DELETE FROM Accounts
        WHERE AccountID = p_AccountID;
    END CloseAccount;
    FUNCTION GetTotalBalance(
        p_CustomerID NUMBER
    ) RETURN NUMBER
    IS
        v_Total NUMBER;
```

```
        BEGIN
            SELECT NVL(SUM(Balance),0)
            INTO v_Total
            FROM Accounts
            WHERE CustomerID = p_CustomerID;

            RETURN v_Total;
        END GetTotalBalance;

END AccountOperations;
/
BEGIN
    AccountOperations.OpenAccount(
        1,
        101,
        'Savings',
        10000
    );
    AccountOperations.OpenAccount(
        2,
        101,
        'Checking',
        5000
    );
    COMMIT;
```

```
SELECT AccountOperations.GetTotalBalance(101) AS Total_Balance
FROM DUAL;
BEGIN
    AccountOperations.CloseAccount(2);

    COMMIT;
END;
```

	TOTAL_BALANCE
1	30